



Gleaming the Cube:

Exploring the limits of Airflow through the Rubik's Cube

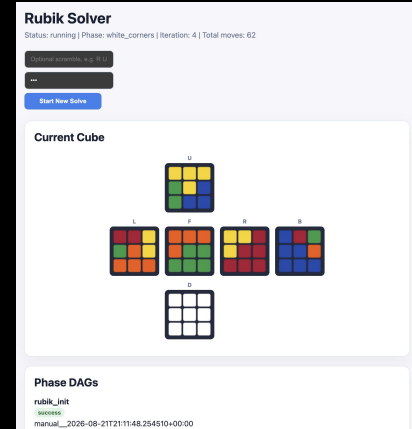
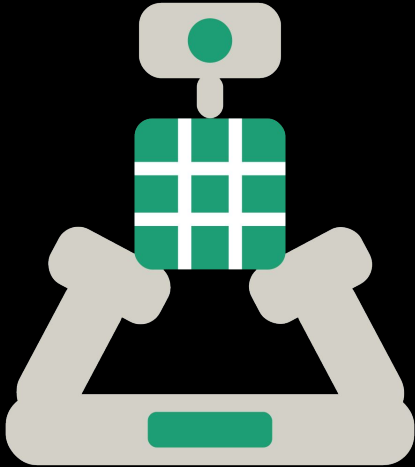


Jonathan Leek, Senior Solution Engineer

Who am I?

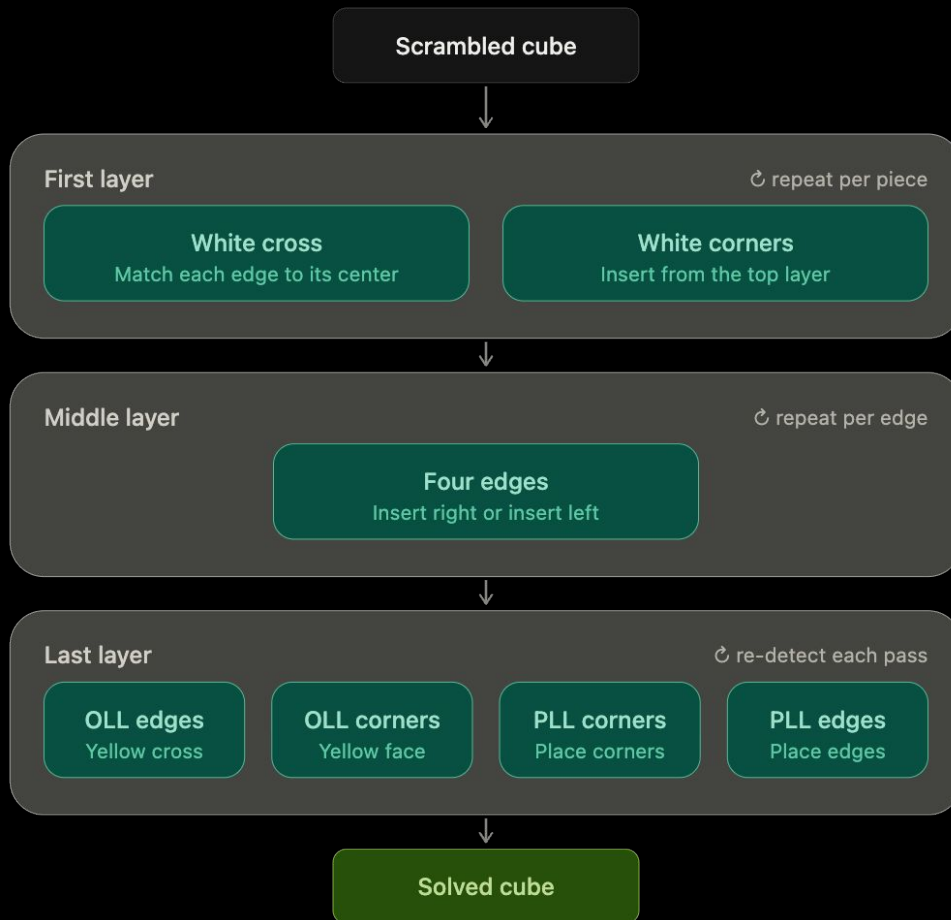


What is this thing?



<https://www.thingiverse.com/thing:3826740>

https://github.com/jonathanleek/Rubik_robot



What makes this hard?



4 Solutions



1. The Right Way
2. The Not So Right Way
3. The Wrong Way
4. The Future Way

The Right Way



```
with DAG(
    dag_id="rubik_library_solver",
    schedule=None,
    start_date=datetime(2026, 8, 23, tz="America/Chicago"),
    catchup=False,
    tags=["rubik", "summit"],
) as dag:
    PythonOperator(
        task_id="solve_cube",
        python_callable=solve_cube,
    )
```

The Not So Right Way



1. Store the state in an Airflow Variable.
2. Break the algorithm steps into separate dags, and use dag runs as the looping mechanism. The dag contains the loop's logic, and at the end of each step, it terminates in one of three ways:
 - a. **Stage is not solved.** Apply the algorithm once, save the new state, and trigger a fresh run of the same dag
 - b. **Stage is solved.** Trigger this dag again.
 - c. **Too many tries.** Stop and fail the dag.

https://github.com/jonathanleek/airflow_rubik_solver



Solid = TriggerDagRunOperator. Dashed = shared state and Asset lineage. ⌚ = trigger_dag_id points at the DAG's own id.
Every phase runs the same shape: read_state → check_phase_solved branch → continue / advance / fail.

RUBIK_SOLVER 
rubik_solve_cross

– 6 TASKS

RUBIK_SOLVER 
read_state

Last Updated 2 months ago ⓘ

RUBIK_SOLVER 
check_phase_solved

Last Updated 2 months ago ⓘ

RUBIK_SOLVER 
apply_algorithm

Last Updated 2 months ago ⓘ

RUBIK_SOLVER 
prepare_next_phase

Last Updated 2 months ago ⓘ

RUBIK_SOLVER 
trigger_self

Last Updated 2 months ago ⓘ

RUBIK_SOLVER 
trigger_next_phase

Last Updated 2 months ago ⓘ

Weird Airflow Consequences



- Idempotency and retries are bad
- Dag runs explode
- `MAX_ITERATIONS` becomes very important
- `wait_for_completion=False` and `max_active_runs=1` are critical

The Wrong Way



Fork Airflow and get weird with it.

<https://github.com/jonathanleek/true-cyclic-dags>

The Wrong Way - Step 1



Create a back-edge that is stored in a separate edge lane, so cycle-validation, trigger rules, and topo-sort never see.

```
apply.loop_to(inspect, until=..., max_iterations=...)  
  
DAG(allow_cycles=True)
```

<https://github.com/jonathanleek/true-cyclic-dags>

The Wrong Way - Step 2



Each pass runs a task group normally, then at `update_state`, the scheduler checks whether the loop should needs to go again, and if it does, it clears the task status of each task in the loop and Airflow runs those steps again.

Repeat until an end condition is met.

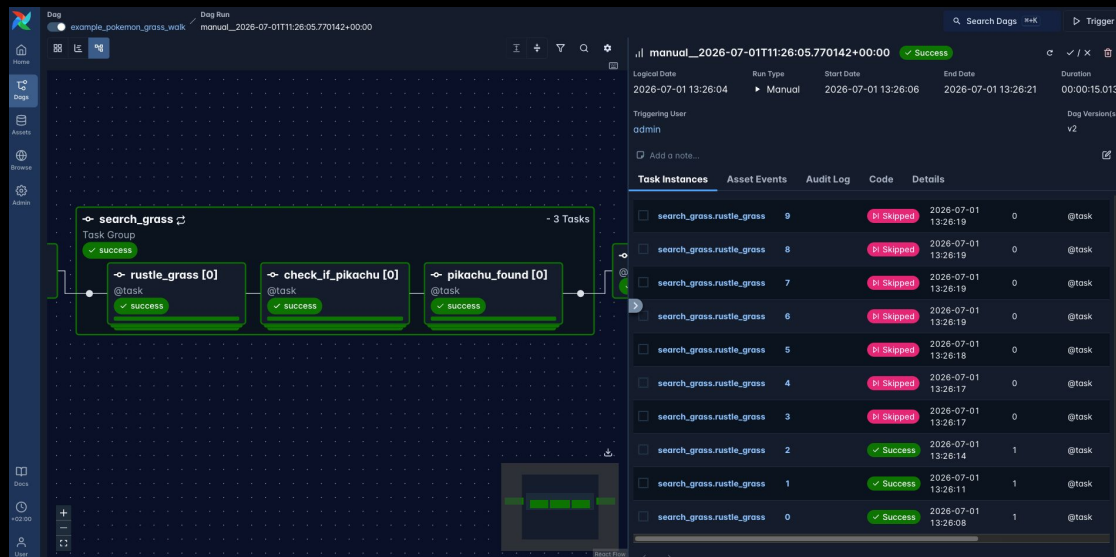
<https://github.com/jonathanleek/true-cyclic-dags>

Why This Is A Bad Idea



- Forking creates an immense maintenance burden
- Grid view only shows the latest pass
- Bad exit guard = neverending dag run
- A better way is coming...

The Future Way - AIP 111



See the talk tomorrow!
Dynamic graphs - Airflow for agentic workflows



Questions?

**Optionally share some contact info like
email, blog or social media handles**